



INFORME EJERCICIO 2



DISEÑO SOFTWARE

Ejercicio realizado por
Sandra Pita Piñón y Candela Naya López

CONTENIDO.

1. Principios de Diseño	1
a. SRP	1
b. OCP	1
c. LSP	2
d. ISP	2
e. DIP	2
2. Patrón de Diseño	2
a. Explicación patrón escogido	2
b. Diagrama de clases	3
c. Diagramas dinámicos	3

PRINCIPIOS DE DISEÑO.

Nos centramos en respetar sobre todo los principios SOLID:

- Principio de responsabilidad unica (SRP).
- Principio abierto-cerrado (OCP).
- Principio de sustitución de Liskov (LSP).
- Principio de segregación de interfaces (ISP).
- Principio de inversión de la dependencia (DIP).

PRINCIPIO DE RESPONSABILIDAD ÚNICA (SRP).

Cada objeto debe tener una responsabilidad unica que esté enteramente encapsulada en la clase. Todos los servicios que provee el objeto están estrechamente alineados con dicha responsabilidad.

En nuestro código separamos cada clase en un archivo .java para resaltar la encapsulación y la responsabilidad. Cada uno tendrá su propio trabajo:

- Expression.java → Interfaz base que define el comportamiento común de cualquier expresión.
- Constant.java → Representa un valor numérico simple.
- UnaryOperation.java → Representa una operación con un único operando.
- BinaryOperation.java → Representa una operación con dos operandos.
- BinaryOperator.java/UnaryOperator.java → Encapsulan la lógica de cada operación matemática.

Esta separación asegura que cada clase haga una sola cosa y esté aislada de cambios ajenos a su responsabilidad.

PRINCIPIO DE ABIERTO - CERRADO (OCP).

Las entidades software (clases, módulos, etc.) deberían ser abiertas para permitir su extensión, pero cerradas frente a la modificación.

En nuestro código, el sistema está abierto a extensión pero cerrado a modificación:

- Podemos añadir nuevos operadores unarios o binarios sin modificar las clases existentes, solo agregando nuevos elementos en los enums.
- Podemos añadir nuevas formas de recorrido, simplemente añadiendo un nuevo Visitor.
- La jerarquía Composite permite añadir nuevos tipos de nodos sin tocar los ya existentes.

Por lo tanto, podemos afirmar que cumple con el Principio de Abierto-Cerrado (OCP).

PRINCIPIO DE SUSTITUCIÓN DE LISKOV (LSP).

Las clases derivadas deben ser sustituibles por sus clases base, es decir, que aquellos métodos que utilicen referencias a clases base deben ser capaces de usar objetos de clases derivadas sin saberlo.

En nuestro código, todas las expresiones (Constant, UnaryOperation, BinaryOperation) pueden ser tratadas como Expression sin romper el comportamiento esperado.

PRINCIPIO DE SEGREGACIÓN DE INTERFACES (ISP).

Muchos interfaces específicos para cada cliente son mejores que un único interfaz de propósito general. El principio de inversión de la dependencia nos conminaba a depender de abstracciones y no de concreciones.

En nuestro código, por ejemplo, la interfaz de ExpressionVisitor define únicamente los métodos necesarios para recorrer una expresión:

- visit(Constant).
- visit(UnaryOperation).
- visit(BinaryOperation).

Además, en nuestro caso, podemos decir que cumplimos este principio debido al ser una interfaz sencilla.

PRINCIPIO DE INVERSIÓN DE LA DEPENDENCIA (DIP).

En este principio, dependemos de las abstracciones y no de las concreciones. La inversión de la dependencia es la estrategia de depender de interfaces o clases y funciones abstractas, en vez de depender de clases y funciones concretas.

En nuestro código, los Visitors dependen de la abstracción Expression, no de sus implementaciones concretas. El código trabaja siempre contra interfaces y clases abstractas. El algoritmo de recorrido nunca depende de operadores concretos, solo de la abstracción UnaryOperator o BinaryOperator.

PATRÓN DE DISEÑO.

EXPLICACIÓN PATRÓN ESCOGIDO.

Para este ejercicio, hemos escogido el Patrón Composición.

PATRÓN COMPOSICIÓN:

- Es un patrón estructural que se utiliza para componer objetos en estructuras de árbol que representan jerarquías todo-parte.
- Este patrón permite tratar uniformemente a los objetos y a las composiciones.
- Lo más importante de este patrón es una clase abstracta que representa al mismo tiempo a los elementos primitivos y a sus contenedores.
- Es un ejemplo de herencia y de composición usadas simultáneamente.
- Algunas de las ventajas de este patrón son:
 - Permitir tratar a objetos básicos y compuestos de manera uniforme, ignorando las diferencias entre composiciones de objetos y objetos individuales.
 - Facilita la entrada de nuevos componentes.
- Como inconveniente, podemos destacar que si queremos restringir qué componentes concretos pueden ir en una determinada composición es necesario añadir comprobaciones en tiempos de ejecución.
- Este patrón nos permite también representar, como se da en este ejercicio, expresiones matemáticas como árboles. Nuestro diseño incluye:
 - Component → Expression.
 - Leaf → Constant.
 - Composite → UnaryOperation, BinaryOperation.

DIAGRAMA DE CLASES.

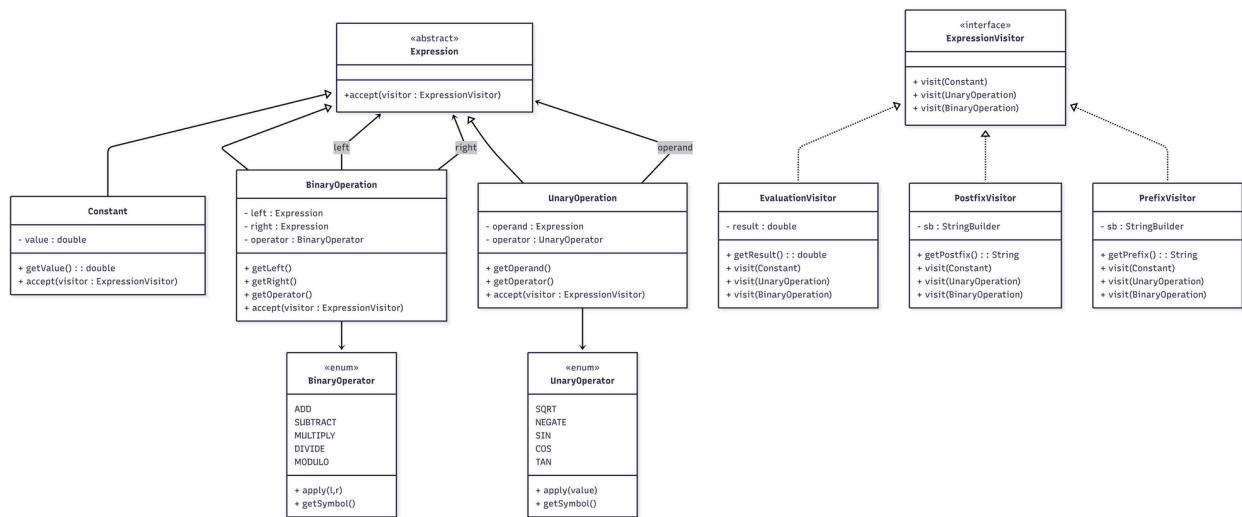
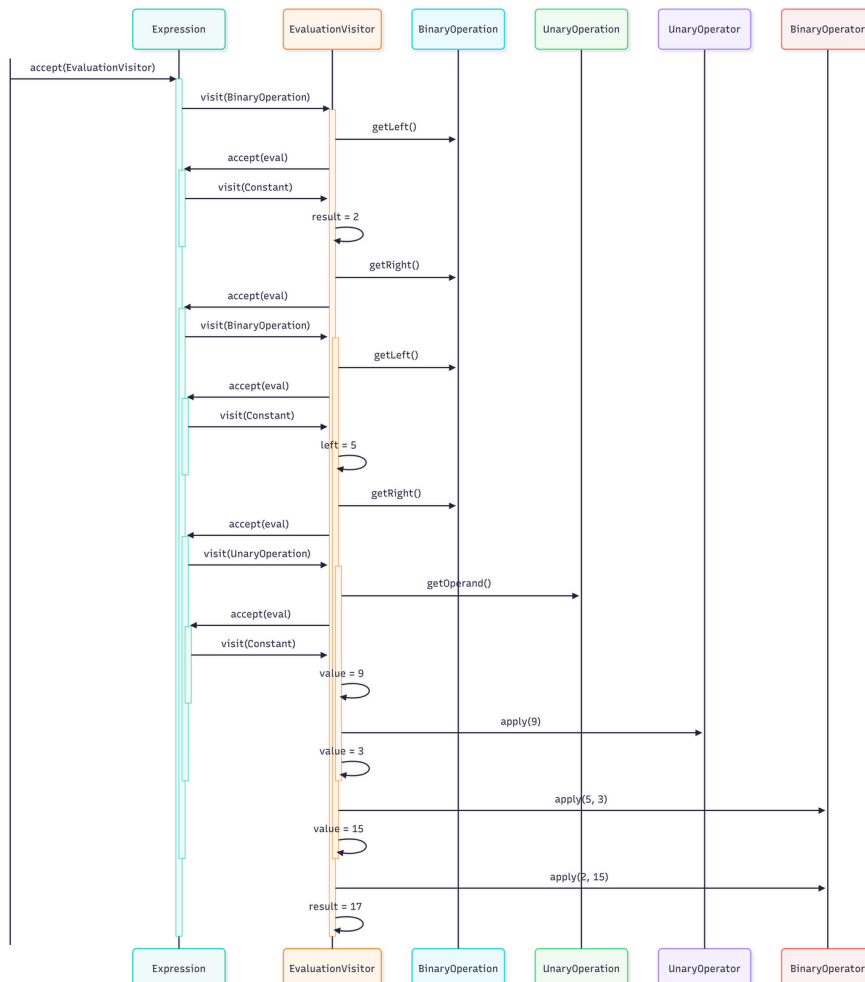


DIAGRAMA DINÁMICO.



El diagrama de secuencia es el diagrama dinámico más adecuado para el ejercicio 2 porque permite representar con claridad el orden de llamadas entre las expresiones y el visitor, muestra la recursión característica del patrón Composite, refleja correctamente la doble llamada accept-visit propia del Visitor y permite describir cómo se evalúa una expresión paso a paso. Otros diagramas dinámicos (como estados o actividades) no representan tan bien la interacción entre objetos ni el flujo de ejecución real.